

AI USAGE DOCUMENT

NL Browser Test Agent

Project Reference · Browser Sandbox QA Automation Platform · Confidential

AI OBJECTIVE

Convert natural language testing instructions into executable browser automation workflows without requiring users to write code.

AI PROBLEM SOLVED

Traditional browser automation requires knowledge of Playwright or Selenium. Non-technical users struggle to create automated tests despite being able to describe workflows in plain English. The AI bridges this gap by understanding user intent and transforming instructions into structured browser actions.

AI Capabilities Used

Natural Language Understanding (NLU)

The AI analyses user-provided test instructions and identifies user intent, browser actions, verification requirements, and execution sequence.

Example Input: "Login as user, add 2 items to cart, checkout, and verify order confirmation."

Extracted Actions:

1. Open Website
2. Login
3. Add Items
4. Checkout
5. Verify Confirmation

Task Planning

After understanding the instruction, the AI / Rule Engine generates a detailed, step-by-step action plan containing ordered browser actions.

Action Mapping

The AI maps interpreted actions to browser automation commands:

`open_url` — Open URL
`click` — Click Element
`fill` — Fill Form
`search_content` — Search Content
`navigate` — Navigate Pages
`add_to_cart` — Add Products to Cart
`checkout` — Checkout

Verification Generation

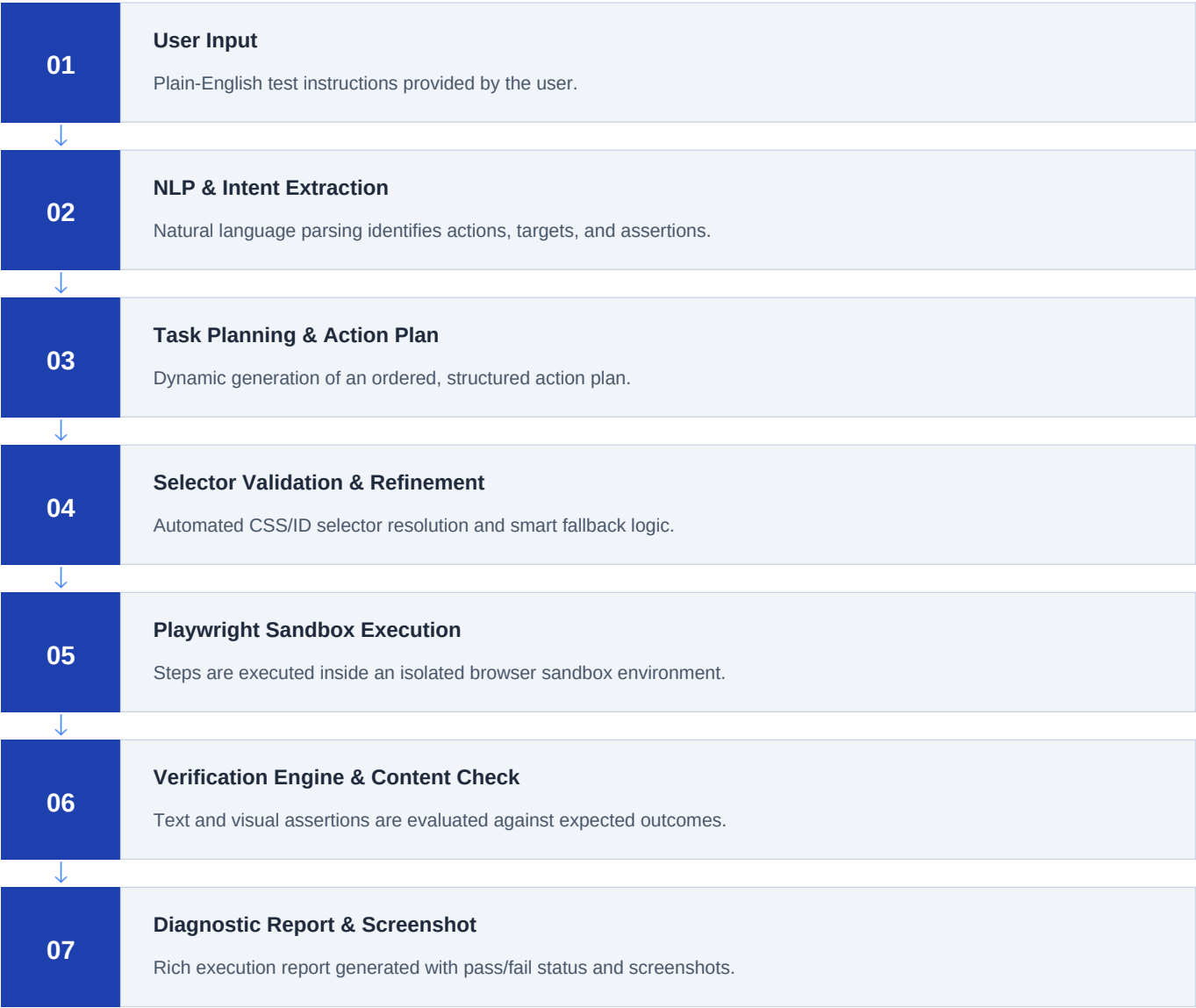
The AI identifies expected outcomes from the user's request and generates validation checks automatically.

1. Verify successful login
2. Verify product added to cart
3. Verify order confirmation message
4. Verify page navigation success

Result Interpretation

After execution, the system analyses browser responses and determines whether each test step has passed or failed, providing diagnostics and warning messages.

AI Workflow



AI Technologies Used

Google Gemini 3.5 Flash Powers high-level suggestion, semantic generation, and deep QA automation pattern reasoning.	Ollama Integration Offers self-hosted / local inference options for enterprise environments requiring data privacy or air-gapped deployments.	Syntactic Rule Engine Compiler High-performance, zero-latency parsing fallback for instantly converting standard demo scenarios (SauceDemo, DemoBlaze) and standard web steps into formal Playwright objects.	Intelligent Selector Resolver Dynamically detects the best CSS/ID selectors — e.g. swapping <code>css=button.btn-primary</code> to <code>css=button[onclick='purchase()']</code> based on application state.
--	---	---	--

Benefits

No Coding Required Empowers product managers, QA analysts, and business stakeholders to generate tests instantly.	Sub-second Performance Optimised compiler guarantees instant step synthesis with fallback stability.
Automatic Validation Rich visual assertions, timing parameters, and retry strategies are computed out of the box.	Failure Tolerance Dynamic first-visible-match locator selection prevents brittle page breaks.

Future Enhancements

- 01

Multi-step Autonomous Exploration
Agents that run interactive crawl loops until finding a target success state.
- 02

Self-healing Test Execution
Automatically repairing broken CSS/XPath selectors based on DOM analysis tree modifications.
- 03

AI-generated Test Coverage Suggestions
Automatically analysing raw application maps to recommend test case scenarios.

Conclusion

The AI integration enables users to create and execute rigorous browser automation tests using natural language. By combining language understanding, modern task planning, robust browser sandboxing, and automated verification, the system significantly reduces the complexity of web application engineering.